ML Engineering & Mathematics of Transformers

Iliad Intensive

Dan Wilhelm (co-authored by Julian Schulz)

November 2, 2026

Outline

1. ML Concepts

2. Mathematics of Transformers

ML Concepts

For parametric models (including neural nets), we follow these steps:

Training Set. From a desired population, we sample N "labeled" examples:

$$\mathcal{D} = \{(x_i, y_i)\}_{i=1}^{N}.$$

Model. For example, $\hat{y}(x; \theta_1, \theta_0) = \theta_1 x + \theta_0$.

Loss. We choose a loss function to minimize how far off our predictions are, e.g. sum of squared errors:

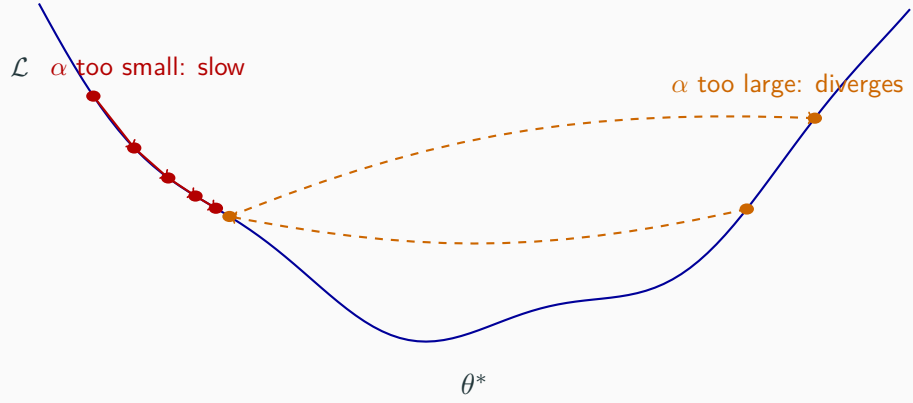
$$SSE(\mathcal{D}) = \sum_{i=1}^{N} (\hat{y}(x_i) - y_i)^2.$$

Optimizer. We choose a method to compute parameter values. Typically gradient descent (e.g. Adam), although we could solve analytically here.

Training. We apply the optimizer to the training set, model, and loss function.

Evaluation. We evaluate the loss again on a held out population sample (test set).

Optimizer: Learning Rate



Iterative weight update rule, for parameter θ, learning rate α, and loss function $\mathcal{L}$:

$$\theta_{t+1} := \theta_t - \alpha \frac{\partial \mathcal{L}}{\partial \theta}(\theta_t).$$

An Example Model: Linear Regression

Let's use gradient descent to minimize a SSE loss function.

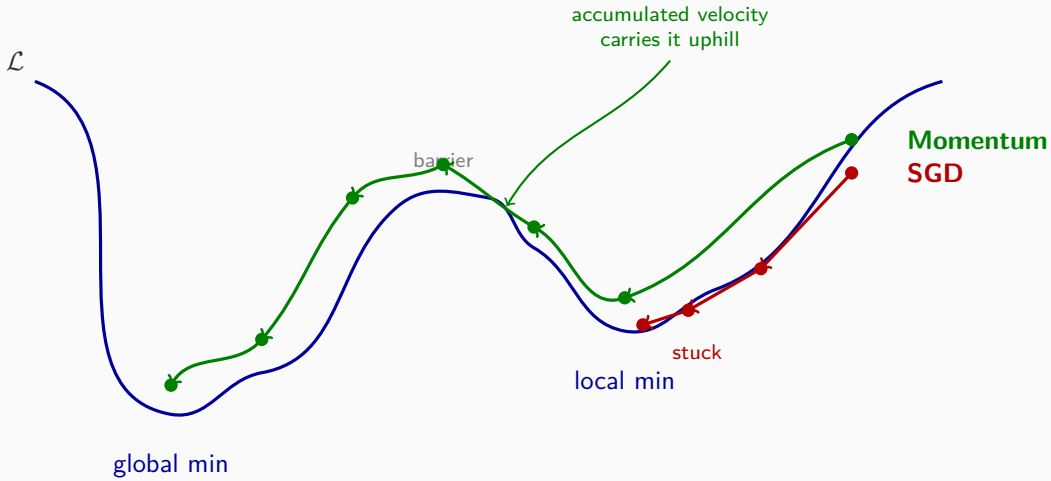
$$\theta_{t+1} := \theta_t - \alpha \frac{\partial \mathcal{L}}{\partial \theta}(\theta_t).$$

```
def L(theta):                  # SSE loss for (1,2) and (2,1)
    return (theta - 2.)**2 + (2*theta - 1.)**2

def dL(w1, delta=1e-3):  # definition of derivative
    return (L(w1 + delta) - L(theta)) / delta

lr = 5e-2        # learning rate
theta = 0.0      # initial weight value
for _ in range(20):
    theta = theta - lr*dL(theta)
    print(theta)        # final value: 0.8 = 4/5
```

Momentum Escapes Local Minima



Adam = Momentum + RMSProp [kingma2014]

$$m_{t+1} = \beta_1 m_t + (1-\beta_1) g_t \qquad v_{t+1} = \beta_2 v_t + (1-\beta_2) g_t^2$$

$$\hat{m} = \frac{m_{t+1}}{1-\beta_1^{t+1}} \qquad \hat{v} = \frac{v_{t+1}}{1-\beta_2^{t+1}}$$

$$\theta_{t+1} = \theta_t - \alpha \frac{\hat{m}}{\sqrt{\hat{v}} + \epsilon}$$

- Adaptive per-parameter learning rates + momentum
- m and v are exponential moving avgs of the gradient g and g^2
- Since these begin at zero, $\hat{m}$ and $\hat{v}$ apply bias correction
- Typical: $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\epsilon = 10^{-8}$

The Training Loop in PyTorch

Here is what this loop looks like in PyTorch:

```
for x, y in dataloader:
    optimizer.zero_grad()          # zero gradients (set dL/dwi = 0)
    prediction = model(x)          # forward pass
    loss = loss_fn(prediction, y) # compute loss
    loss.backward()                # backward pass (adds to dL/dwi)
    optimizer.step()               # update params: wi -= lambda * dL/dwi
```

This is *the* loop, even used for training LLMs.
Everything else in ML is about making each piece better.

Softmax

How might neural networks perform classification? Let's dedicate an output layer neuron for each output class. Then, we could simply select the output class corresponding to the largest output neuron value (i.e. the argmax).

However, argmax is not differentiable hence can't be used with gradient descent. So, we use softmax σ which assigns a probability to each of K output classes, given the K output neuron values ($\sigma : \mathbb{R}^K \to (0,1)^K$):

$$\sigma(\mathbf{z})_i = \frac{e^{z_i}}{\sum_{j=1}^{K} e^{z_j}}$$

Softmax is also applied in transformers and has many important properties that we'll explore in the exercises.

Optional Math Exercises

1. Why might a tanh activation function cause gradient descent to converge more quickly? (Hint: `https://www.wolframalpha.com/input/?i=derivative+tanh`)
2. Softmax. Prove that softmax σ is invariant under translation, i.e. $\sigma(\mathbf{x}) = \sigma(\mathbf{x} + c)$ for any $c \in \mathbb{R}$. Is it invariant under scaling?
3. Softmax. What is $\sigma(\mathbf{c})$, for any k-dimensional $\mathbf{c} \in \mathbb{R}^k$?
4. Softmax. Suppose for some index i, $\sigma(\mathbf{x})_i \approx 0$. What is $\mathbf{x}_i$?

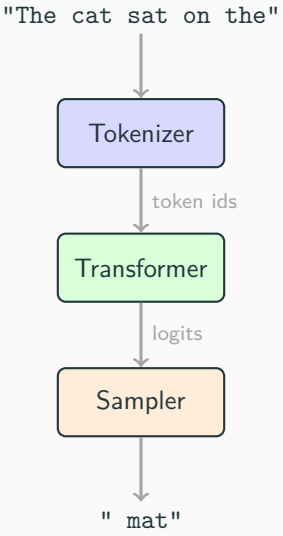
Mathematics of Transformers

Learning Objectives

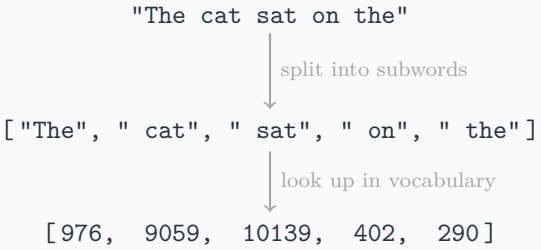
By the end of Mathematics of Transfomers, you will be able to:

1. Draw how the residual stream is modified throughout a transformer.
2. Use equations to describe how each MLP neuron outputs a particular direction to the residual stream.
3. Describe the purpose of LayerNorms and where they are applied in a transformer.
4. Explain how token sequences are embedded into vectors, and how vectors are unembedded into tokens.
5. Show how attention patterns describe what attention incorporates into the residual stream.

A Top-Down View



Tokenizer


"The cat sat on the"
split into subwords
["The", " cat", " sat", " on", " the"]
look up in vocabulary
[976, 9059, 10139, 402, 290]

Building the Tokenizer: Byte Pair Encoding (BPE)

Start with the vocabulary as all single characters (i.e. bytes, note that Unicode/emojis can require 2-4 bytes!), then **repeat**:

1. Find the most frequent adjacent pair.
2. Merge it into a new symbol; add it to the vocabulary.

Stop at the target vocabulary size.

Sampler

The transformer outputs one **logit** per vocabulary token: $l \in \mathbb{R}^{|V|}$. To do so, the sampler scales each logit by the temperature T then takes the softmax:

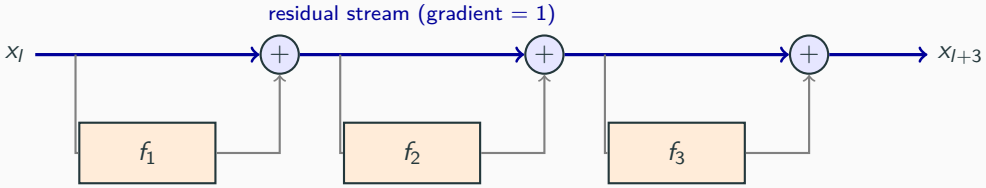
$$p_i = \text{softmax}(l_i/T) = \frac{e^{l_i/T}}{\sum_j e^{l_j/T}}$$

Temperature T: as $T \to 0$ this recovers greedy ($\arg\max$); $T < 1$ sharpens the distribution, $T > 1$ flattens it.

- **Top-k**: keep the k highest-probability tokens, renormalize, then sample.
- **Top-p** (nucleus): keep the smallest set of tokens whose probabilities sum to at least p, renormalize, then sample.

Deep Learning: Residual Connections

Deep networks suffer from **vanishing gradients**: gradients shrink to zero through many layers due to ever-increasing multiplications.

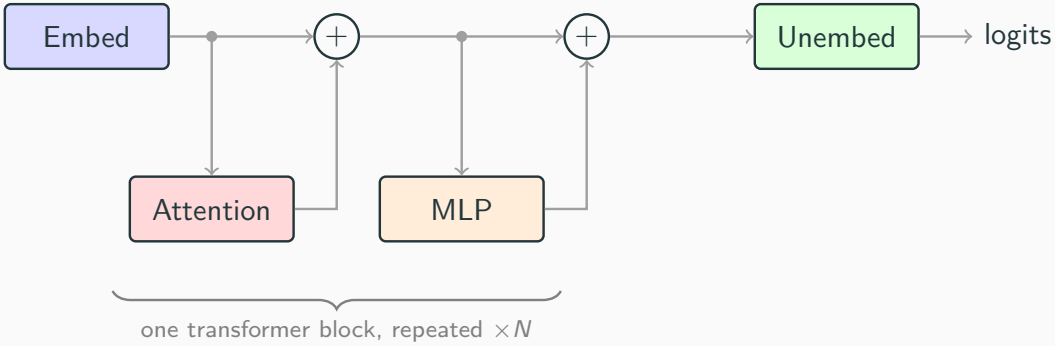


$$x_{l+1} = x_l + f(x_l)$$

- Gradient flows unimpeded through the skip path (identity)
- Each block only needs to learn the *residual* correction $f(x_l)$

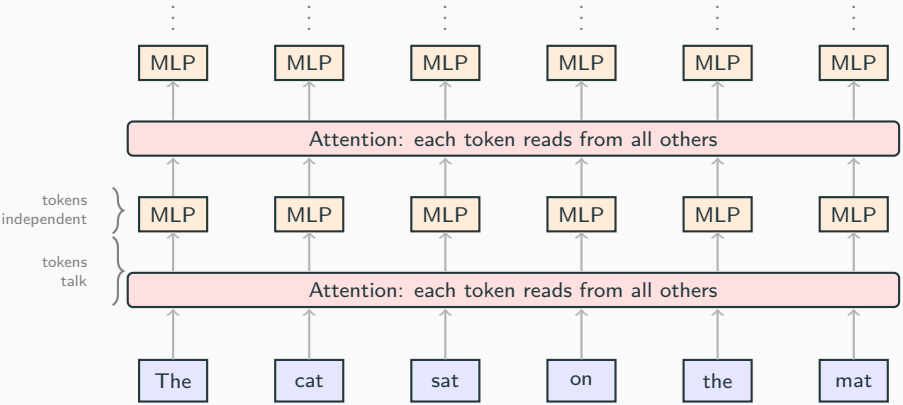
[he2015]

Inside the Transformer



- **Attention head**: moves information between token positions.
- **MLP**: stores / computes information.
- **Residual stream**: begins as the token embeddings and passed layer to layer, summed with each attention and MLP output.

Transformers (Preview)



- **Attention**: tokens mix information (the only place they interact)
- **MLP**: each token processed independently (same weights, different data)

[vaswani2017]

Embedding

Given vocabulary size $|V|$, each token id selects a single row from a lookup table $W_E \in \mathbb{R}^{|V| \times d}$. This row is a learned vector that encodes the token's meaning.

$$\text{token id } t \longmapsto x_t = (W_E)_t \in \mathbb{R}^d$$

The vectors are **learned**, so directions in this space come to encode semantic relationships.

Positional Encoding

Learned (absolute)

A second lookup table, one learned vector per position:

$$P \in \mathbb{R}^{L \times d}, \qquad \text{pos} \mapsto P_{\text{pos}}$$

$$x_{\text{pos}} = \text{embed}(\text{token}) + P_{\text{pos}}$$

The obvious default; capped at a maximum length L, and position is **absolute**.

Rotary (RoPE)

Rotate each 2D pair of query/key dims by an angle $\propto$ position:

$$\tilde{q}_m = R(m\theta)\, q, \qquad \tilde{k}_n = R(n\theta)\, k$$

$$R(\alpha) = \begin{pmatrix} \cos\alpha & -\sin\alpha \\ \sin\alpha & \cos\alpha \end{pmatrix}$$

$$\langle \tilde{q}_m, \tilde{k}_n \rangle = g(q, k,\, m - n)$$

Score depends only on the **relative** offset $m - n$; applied every layer.

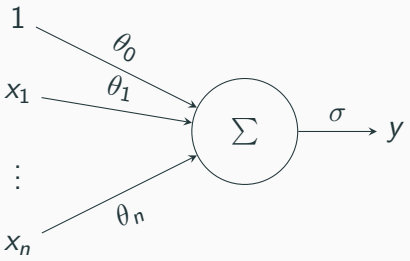
Unembedding

Not the inverse of embedding, unless the embeddings are tied (e.g. GPT-2).

$$\text{logits} = X\, W_U, \qquad W_U \in \mathbb{R}^{d \times |V|}$$

- Reads the final residual stream and projects it back to vocabulary size.
- Produces one logit per token, ready for the sampler.
- Uses its own learned weights W_U, separate from W_E (unless tied).

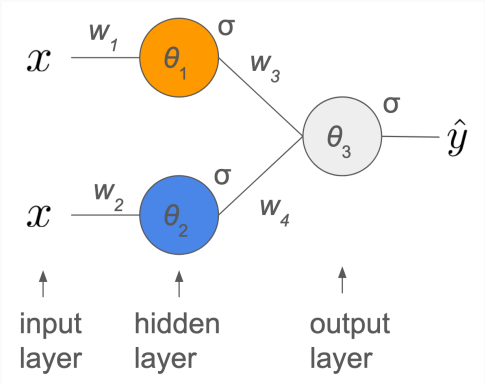
A Neuron



A neuron's activation is the dot product of an input vector $\mathbf{x}$ (with prefixed 1) and weight vector θ. For some activation function σ:

$$y(\mathbf{x}; \theta) = \sigma(\mathbf{x} \cdot \theta).$$

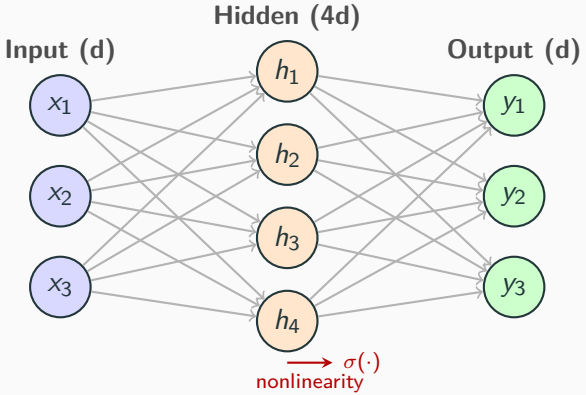
Regression or Classification: Neural Networks



- A neural network is a composition of neurons.
- Its diagram visually depicts an equation/model.
- Each neuron can be assigned a distinct activation function.

$$\hat{y}(x; \mathbf{w}, \theta) = \sigma(\sigma(w_1 x + \theta_1) w_3 + \sigma(w_2 x + \theta_2) w_4 + \theta_3)$$

The MLP: Multi-Layer Perceptron



- Transformers typically have $4d$ MLP neurons (for empirical reasons).
- Given activation function σ and nxd input X, the MLP neuron activations are the nx$4d$ matrix $X' = \sigma(XW_{\text{in}} + b_{\text{in}})$, and $\text{mlp_out} = X'W_{\text{out}} + b_{\text{out}}$.

Interpreting the MLP

The MLP acts on each residual stream row independently. It does not have access to prior rows.

Each MLP neuron i contributes a fixed direction to the residual stream, row i of W_{out}. This direction is multiplied by the activation strength, which is effectively how well the input row i "matches" with column i of W_{in}.

LayerNorm

In Attention, MLP, and pre-unembedding, the input is first normalized. This assists with learning, since the transformer no longer must worry about input scale.

For each row of the residual stream, the LayerNorm replaces each dimension with its "z-score" (number of standard deviations from the mean). Each dimension is individually scaled by $\gamma \in \mathbb{R}^d$ and translated by $\beta \in \mathbb{R}^d$:

$$y = \frac{x - E[x]}{\sqrt{\text{Var}[x] + \epsilon}} * \gamma + \beta$$

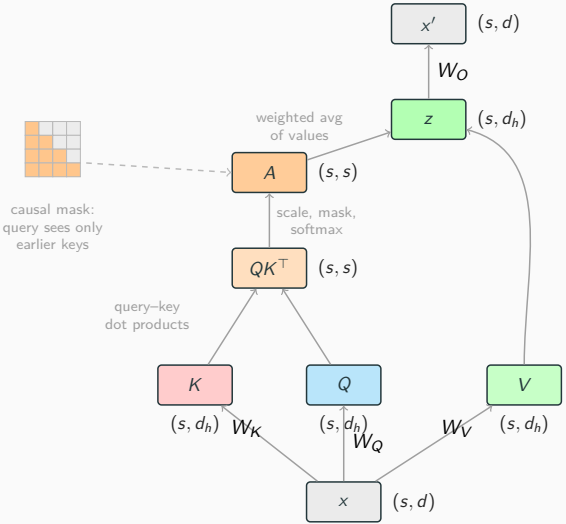
Note that some small ϵ (e.g. 1e-5) ensures no division by zero.

LayerNorm: https://arxiv.org/abs/1607.06450

Attention

Attention is the only part of the transformer where we look back

Anatomy of an Attention Head



$$\text{Attention}(Q, K, V) = \text{softmax}\Big(\frac{QK^\top}{\sqrt{d_h}}\Big)V \quad | \quad n\text{: seq length, } d\text{: model dim, } d_h\text{: head dim}$$

Attention: KQV

First, we compute the key (K), query (Q), and value (V) matrices from the (normalized) residual stream X'. These are simply linear transforms, e.g.

$$K = X'W_K + b_K \tag{1}$$
$$Q = X'W_Q + b_Q \tag{2}$$
$$V = X'W_V + b_V \tag{3}$$

The idea is that we will assess how similar the keys are to the queries, then pay attention to those with highest similarity.

Attention: Pattern

The dot product can be used as a measure of similarity (in fact, it is un-normalized cosine similarity!). So, we compute the similarity of each key with each query by taking each of their dot products (for a single head, QK^T; for scaling softmax, we divide by $\sqrt{d_h}$, the dimensionality of each head).

Larger scores indicate they are more "similar", hence we are effectively matching keys with queries (similarly to a database lookup). We now have nxn scores, one for each pair of sequence tokens.

We finally take the per-row softmax. This results in the attention pattern, where each row i (corresponding to token i) sums to 1. We typically use "causal attention" where row i sets columns $> i$ to -infinity, ensuring the softmax returns 0 (no attention paid to tokens following i).

Attention Output

The main idea about causal attention is that for each residual stream row, it takes a weighted sum of the prior rows in V, essentially paying "attention" to them. Everything else is minor details concerning how those weights are computed (and a final linear transform).

Supposing a single head, attn_out is then simply a linear transform of this weighted sum:

$$\text{attn_out} = (PV)W_O + b_O.$$

Specifically, the $n \times n$ weight matrix P is the "attention pattern". Each row sums to 1 (due to softmax), and for row i all columns $> i$ are 0 (since we cannot attend to rows following it).

Multi-Headed Attention Output

Most transformers have multiple heads. These compute outputs in parallel, then we sum each of their outputs with b_O to obtain attn_out.

Given residual stream dimension d, the W_Q^h, W_K^h, and W_V^h transforms typically map dimension d to $d_h = \left\lfloor \frac{d}{n_h} \right\rfloor$ (where n_h is the number of heads).

We obtain for each head a distinct n×n attention pattern P^h, n×d_h value matrix V^h, then apply the d_h×d per-head transform W_O^h:

$$\text{attn_out} = \sum_{h=1}^{n_h} (P^h V^h W_O^h) + b_O.$$

Optional Math Exercises

1. Prove that two affine transformations can be rewritten as a single affine transformation, i.e. for $f(x) = Ax + b$ and $g(x) = Cx + d$, what is $g(f(x))$? (this is why activation functions must be non-linear!)
2. Why can we mean-center the unembedding columns (W_U) without affecting token probabilities?
3. Expand QK^T in terms of $W_Q W_K^T$. What might be the benefits of factorizing the parameters $W_Q W_K^T$ into separate matrices W_Q and W_K?
4. Can you prove that b_K has no effect on the attention pattern probabilities?
5. In the transformer, there are linear transforms immediately following each other. We can combine these into single linear transforms. Where are they? And why might we not have combined them earlier?

Optional Exercise: Implement Optimizers

25 minutes



Optional Coding Exercises: Implement MLP/Attention

MLP



Attention

